

로봇 트윈 사용 설명서

ROS 2 를 직접 쓰기 어려운 환경에서 **HTTP/JSON 만으로 로봇 상태를 조회하고 제한된 제어를 수행**하기 위한 게이트웨이이다. 클라이언트는 언어를 가리지 않는다 — curl, Java, Python, Node-RED 모두 같은 인터페이스를 쓴다.

- 설계 근거와 결정 이력: [robot_twin_design.md](#)
- 프로토콜 원본: `extern_op` (RESTful 기반 외부 시스템 연동 프로토콜)

⚠ 보안 전제: 현재 트윈은 **인증·권한·TLS** 를 제공하지 않는다. 이 API 에 라우팅이 닿는 주체는 누구든 로봇을 움직일 수 있다. 반드시 **신뢰된 폐쇄망** 안에서만 운용한다 (9장 참조).

1. 빠른 시작

1.1 사전 조건

```
# pip 전용 의존성 (rosdep 이 잡지 못한다)
pip install --user 'fastapi' 'uvicorn[standard]' 'pydantic>=2'

# 선택: 연산 입력의 JSON Schema 검증을 켜려면
pip install --user 'jsonschema'
```

빌드는 워크스페이스 루트에서 한다.

```
cd ~/development/ros/rdfp_ws
colcon build --packages-select rdfp_msgs robot_control robot_twin rdfp
source install/setup.bash
```

1.2 로봇 스택 기동

트윈은 로봇을 제어하지 않는다. **MoveIt2 스택이 먼저 떠 있어야 한다.**

```
ros2 launch robot_control panda_mock.launch.py
```

이것으로 충분하다. 트윈은 **제어 계층**에 속하고 `package.xml` 의 의존도 `robot_control` 뿐이므로, `arm.gripper.scene` 연산은 이 스택만으로 전부 동작한다. 썬 노드(`reset_scene` 이 필요로 한다)도 이 `launch` 가 기본으로 함께 띄운다 (`enable_scene_node:=false` 로 끌 수 있다).

세션/에피소드 연산(`start_session` 등)을 쏠 때만 수집 계층 `launch` 로 바꾼다 — 그 연산들의 실제 구현은 `rdfp` 의 `session_control_node` 이고, 트윈은 그것을 `entry point` 로 선택적으로 붙이기 때문이다. 노드가 없으면 해당 연산만 `PRECONDITION_FAILED` 로 거부되고 나머지는 정상 동작한다.

```
ros2 launch rdfp rdfp_panda_mock.launch.py    # 세션/에피소드 연산까지 쏠 때
```

1.3 트윈 시작

```
ros2 run robot_twin robot_twin --config /절대/경로/src/robot_twin/config/robot_twin_panda01.yaml
```

⚠ **--config 는 절대 경로를 쓴다.** `ros2 run` 은 작업 디렉터리를 보장하지 않으므로 상대 경로는 `twin config not found` 로 실패한다. 설치본을 쓰려면:

```
ros2 run robot_twin robot_twin \
  --config "$(ros2 pkg prefix robot_twin)/share/robot_twin/config/robot_twin_panda01.yaml"
```

정상 기동 로그는 다음 순서로 나온다. 이 순서 자체가 설계다 (2장 참조).

```
robot twin 'panda01' starting (move_group_mode=jtc, http=0.0.0.0:8801)
variable 'joint_states' <- /joint_states (sensor_msgs/msg/JointState, reliable/volatile)
variable 'ee_pose' <- /ee_pose (geometry_msgs/msg/PoseStamped, reliable/volatile)
...
no authentication or TLS is configured; binding 0.0.0.0:8801 ← 경고는 정상이다
Uvicorn running on http://0.0.0.0:8801
MoveGroup client ready (mode=jtc) ← 여기부터 연산 가능
```

1.4 동작 확인

```
curl -s http://127.0.0.1:8801/api/v1/robot_twins/panda01/health | python3 -m json.tool
```

`move_group` 이 `READY` 이면 연산을 받을 수 있다.

1.5 종료

Ctrl+C 로 종료한다. 백그라운드로 띄웠다면 **프로세스 그룹 단위**로 종료해야 자식 프로세스가 고아로 남지 않는다.

```
# PGID 확인 후 그룹 전체에 SIGTERM
ps -eo pid,pgid,args | grep 'lib/rdfp/robot_twin' | grep -v grep
kill -TERM -<PGID>
```

종료 시 주의: 실행 중인 연산이 있으면 로봇이 경로 중간에 멈추고, 클라이언트는 이후 세션 조회에서 **404** 를 받는다. 안전하게 내리려면 먼저 실행 중 세션이 없음을 확인한다.

```
curl -s '.../operations/sessions?status=RUNNING'
```

1.6 대화형 API 문서

FastAPI 가 자동 생성한다. 브라우저로 열면 모든 엔드포인트를 직접 호출해 볼 수 있다.

URL	용도
http://<host>:8801/docs	Swagger UI
http://<host>:8801/openapi.json	OpenAPI 스펙 (클라이언트 스텝 생성용)

Java 클라이언트가 필요하면 스펙에서 바로 뽑을 수 있다.

```
curl -s http://127.0.0.1:8801/openapi.json > twin-openapi.json
openapi-generator generate -i twin-openapi.json -g java -o ./twin-client-java
```

2. 아키텍처 한눈에

```
[REST 클라이언트] —HTTP/JSON—▶ [로봇 트윈 (Python 단일 프로세스)] —DDS—▶ [ROS 2 / MoveIt2]
                                │ FastAPI (HTTP 스레드)
                                │ 상태 스냅샷 캐시
                                └─ rclpy executor (별도 스레드)
```

핵심 성질 두 가지를 알고 쓰면 오해가 없다.

1. **상태 조회는 캐시를 읽는다.** 트윈이 토픽을 상시 구독해 최신값을 들고 있다가 반환한다. 조회 시점에 ROS 에 묻지 않으므로 빠르지만, **값에는 나이가 있다.**
2. **1 트윈 = 1 프로세스 = 1 로봇.** ROS_DOMAIN_ID 가 프로세스 환경변수라서다. 여러 대를 다루려면 프로세스를 여러 개 띄운다 (7장).

3. 클라이언트 인터페이스

3.1 엔드포인트 전체

{twin} 은 설정의 twin.id 다 (예제에서는 panda01).

```

# 디스커버리
GET    /api/v1/robot_twins                                트윈 목록 (항상 1개)
GET    /api/v1/robot_twins/{twin}                         메타 + 카탈로그
GET    /api/v1/robot_twins/{twin}/health                 ROS 연결·준비 상태

# 상태 변수
GET    /api/v1/robot_twins/{twin}/variables              변수 목록
GET    /api/v1/robot_twins/{twin}/variables?names=a,b   배치 조회
GET    /api/v1/robot_twins/{twin}/variables/{name}     단일 조회
GET    /api/v1/robot_twins/{twin}/variables/{name}?refresh=true 정적 소스 강제 갱신
HEAD   /api/v1/robot_twins/{twin}/variables/{name}       ETag 만 확인

# 자원 점유
GET    /api/v1/robot_twins/{twin}/resources            arm/gripper 점유 상태

# 연산
GET    /api/v1/robot_twins/{twin}/operations          연산 카탈로그
POST   /api/v1/robot_twins/{twin}/operations/{op}        실행
GET    /api/v1/robot_twins/{twin}/operations/{op}/sessions 해당 연산의 세션 목록
GET    /api/v1/robot_twins/{twin}/operations/sessions 전체 세션 (?status=RUNNING)
GET    /api/v1/robot_twins/{twin}/operations/sessions/{sid} 세션 상태 조회
DELETE /api/v1/robot_twins/{twin}/operations/sessions/{sid} 세션 취소

# 안전
POST   /api/v1/robot_twins/{twin}/estop                비상정지
DELETE /api/v1/robot_twins/{twin}/estop                비상정지 해제

```

3.2 상태 변수 조회 — 응답 형식

```

GET /api/v1/robot_twins/panda01/variables/joint_states

200 OK
ETag: "1043"
Cache-Control: no-cache, must-revalidate
{
  "name": "joint_states",
  "quality": "OK",
  "stamp": { "sec": 1786187482, "nanosec": 326478824 }, // 로봇이 찍은 시각
  "received_at": "2026-08-08T11:11:22.412Z",           // 트윈이 받은 시각
  "age_ms": 11,
  "schema_version": 1,
  "value": {
    "header": { "stamp": { "sec": 1786199567, "nanosec": 168720085 },
                "frame_id": "base_link" },
    "position": { "panda_joint1": 0.0, "panda_joint2": -0.785, "...": 0.0 },
    "velocity": { "panda_joint1": 0.0, "...": 0.0 },
    "effort": { "panda_joint1": null, "...": null }
  }
}

```

value 는 **원본 ROS 메시지를 규약에 따라 변환한 JSON** 이다. 메시지의 모든 필드가 들어가므로 header 도 포함된다. 필요한 필드만 골라 쓰면 된다.

응답 전체가 JSON 이므로 별도 파서가 필요 없다 — Content-Type: application/json .

값 변환 규약 (알아두면 놀라지 않는다)

ROS	JSON	왜
float NaN / Inf	null	JSON 에 해당 리터럴이 없다. 위 effort 가 실제 사례 — mock 은 토크를 발행하지 않아 전부 null 이다
name[] + position[] 병렬 배열	{관절이름: 값} map	인덱스 순서 의존은 컨트롤러 설정이 바뀌면 조용히 틀린다
정수 enum 상수	문자열 심볼 + <필드>_value	클라이언트가 매직넘버를 하드코딩하지 않도록
builtin_interfaces/Time	{"sec": ..., "nanosec": ...}	ISO8601 로 바꾸면 정밀도가 손실된다
int64 / uint64 (2^53 초과)	문자열	JS Number 정밀도 손실 방지
byte array	노출하지 않음	이미지 등은 상태 변수로 두지 않는다

단위 변환은 하지 않는다. m·rad 원본 그대로다.

반드시 quality 를 확인하고 쓴다. 값만 읽으면 낡은 값을 최신으로 오인한다.

quality	의미	클라이언트 조치
OK	값이 신선하다	그대로 사용
STALE	값은 있으나 임계값보다 오래됐다	사용 여부를 판단. 발행이 끊겼을 수 있다
NO_DATA	아직 한 번도 수신하지 못했다	로봇 스택이 기동 중이거나 미배선 소스다
SOURCE_UNAVAILABLE	소스 자체가 없다	reason 확인. 대개 노드 미기동
ERROR	수신했으나 변환에 실패했다	error 확인. 설정·메시지 불일치

SOURCE_UNAVAILABLE 의 reason : NO_PUBLISHER / TF_LOOKUP_FAILED / SERVICE_UNAVAILABLE / SOURCE_NODE_DOWN .

404 는 "그런 변수가 정의되어 있지 않다"에만 쓴다. 변수는 있고 값만 없는 상황은 200 + quality: "NO_DATA" 다.

폴링 대역폭 줄이기 — ETag

값이 바뀌지 않았으면 304 로 끝난다. 본문이 오지 않으므로 고빈도 폴링에 효과가 크다.

```
ETAG=$(curl -s -D- -o /dev/null .../variables/joint_states | grep -i '^etag' | cut -d' ' -f2)
curl -s -o /dev/null -w '%{http_code}\n' -H "If-None-Match: $ETAG" .../variables/joint_states
# 값이 그대로면 304
```

정적 소스 강제 갱신 — ?refresh=true

source.type: static 인 변수(named_targets)는 최초 조회 시 한 번 가져와 무기한 캐시한다. 자동 무효화를 하지 않으므로 move_group 이 재시작되어 SRDF 가 바뀌어도 옛 값이 계속 나온다. 갱신 수단은 두 가지뿐이다.

```
curl -s '.../variables/named_targets?refresh=true' # 캐시를 버리고 다시 조회
# 또는 트윈 재시작
```

다른 소스 타입에는 아무 영향이 없다. 갱신에 실패해도 **이미 캐시된 값은 그대로 유지된다** — 값이 런타임에 변하지 않는다는 정적 소스의 전제 때문이다.

배치 조회 — 보장 수준에 주의

```
curl -s '.../variables?names=joint_states,ee_pose'
```

보장하는 것은 **"같은 시점에 읽은 각 변수의 최신값"** 이다. **각 값이 같은 시각에 생성되었다는 뜻이 아니다**. 두 값은 서로 다른 노드가 다른 주기로 발행한 것이라 `stamp` 가 일치하지 않는다. 정합이 필요하다면 클라이언트가 `stamp` 를 직접 비교한다.

3.3 연산 실행 — 동기와 비동기

연산마다 **동기/비동기가 고정**되어 있다. `GET .../operations` 의 `kind` 로 미리 알 수 있으므로 클라이언트가 런타임에 분기할 필요가 없다.

동기 연산 — 200 + 종료 상태로 한 번에 끝난다.

```
POST /api/v1/robot_twins/panda01/operations/move_gripper_to_target
{ "inputs": { "target": "open" } }

200 OK
{ "status": "COMPLETED",
  "outputs": { "target": "open",
               "position": 0.0399,    // 액션 result - 실제 도달 위치 [m]
               "effort": 0.0,
               "stalled": false,     // 물체를 몰어 멈췄는가
               "reached_goal": true }} // 목표 위치에 도달했는가
```

비동기 연산 — 202 + `session_endpoint` 를 받고 폴링한다.

```
POST /api/v1/robot_twins/panda01/operations/move_to_named_target
{ "inputs": { "target": "ready", "velocity_scaling": 0.3 } }

202 Accepted
Location: /api/v1/robot_twins/panda01/operations/sessions/9e024bc5700c
{
  "status": "RUNNING",
  "phase": "PLANNING",
  "session_endpoint": "/api/v1/robot_twins/panda01/operations/sessions/9e024bc5700c"
}
```

이후 `session_endpoint` 를 폴링한다.

GET /api/v1/robot_twins/panda01/operations/sessions/9e024bc5700c

```
200 OK
{
  "status": "COMPLETED",
  "outputs": {
    "final_pose": { "...": "..."},
    "final_joints": { "...": "..."},
    "closed_loop": true,
    "measured_age_ms": 21
  }
}
```

상태와 국면

status	의미
RUNNING	실행 중
COMPLETED	성공 종료
FAILED	실패 종료. error.code 로 사유 구분
CANCELLED	클라이언트 요청으로 취소됨. error 가 붙지 않는다

phase 는 RUNNING 안의 국면이다: PLANNING → EXECUTING , 취소 중이면 CANCELING . 계획에 수 초가 걸릴 수 있으므로, phase 를 보면 "멈춰 있는 것"과 "계획 중인 것"을 구분할 수 있다.

CANCELLED 와 FAILED 를 혼동하지 않는다. E-stop 이나 위치독으로 중단된 것은 클라이언트 요청이 아니므로 FAILED + ESTOP_ENGAGED / TIMEOUT 이다.

3.4 오류 응답

HTTP 상태 코드는 프로토콜 처리 결과, 본문의 status 는 연산 결과다.

세션이 만들어진 뒤의 실패 — 200 + status: FAILED

error.code	발생 조건
PLANNING_FAILED	IK 실패 / 충돌 / 도달 불가
EXECUTION_ABORTED	컨트롤러 tolerance 위반, 실행 중 오류
TIMEOUT	max_duration_sec 초과로 위치독이 중단
ESTOP_ENGAGED	실행 중 비상정지로 중단
PREEMPTED	다른 세션에 선점됨

세션이 만들어지지 않은 거부 — 4xx / 5xx. status 가 없다.

HTTP	error.code	의미
400	INVALID_INPUT	입력 검증 실패. 로봇에 닿기 전에 걸렸다
404	NOT_FOUND	없는 변수·연산

HTTP	error.code	의미
404	SESSION_NOT_FOUND	세션 만료·축출·트윈 재시작
409	RESOURCE_BUSY	자원 점유 중. Retry-After 참고
409	ESTOP_ENGAGED	비상정지 잠금 상태
503	PRECONDITION_FAILED	그 연산이 쓰는 백엔드가 미준비. 현재는 MoveGroup 클라이언트뿐이며, 팔 연산에만 적용된다

409 RESOURCE_BUSY 는 점유자를 함께 알려준다.

```
409 Conflict
{ "error": {
  "code": "RESOURCE_BUSY",
  "message": "resource 'arm' is occupied by move_to_named_target",
  "resource": "arm",
  "operation": "move_to_named_target",
  "occupied_by": "/api/v1/robot_twins/panda01/operations/sessions/9e024bc5700c" } }
```

현재 estimated_remaining_ms 와 Retry-After 는 오지 않는다. 잔여 시간 산출에 필요한 꺾적 duration 을 백엔드가 아직 채우지 않기 때문이다(11장). 응답에 두 값이 있으면 쓰고, 없으면 **클라이언트가 자체 백오프**를 적용한다.

큐잉하지 않는다. 순차 실행이 필요하면 클라이언트가 COMPLETED 를 확인하고 다음을 호출한다. 재시도 시 **jitter 를 반드시 더한다** — 여러 클라이언트가 같은 시점에 재시도하면 몰린다.

세션 404 를 완료로 가정하지 않는다 ⚠

폴링 중 404 가 나오면 대개 트윈이 재시작된 것이다. 이때 연산의 최종 결과는 알 수 없다. **완료로 가정하지 말고 상태 변수로 로봇의 실제 위치를 확인한다.**

4. 제공되는 상태 변수와 연산

4.1 상태 변수

스택 열의 의미는 4.2 와 같다 — **제어**는 robot_control panda_mock 만으로, 수집은 rdfp rdfp_panda_mock 이 있어야 값이 온다.

변수	스택	소스	상태	설명
joint_states	제어	/joint_states	✔	관절 위치·속도·토크. {관절이름: 값} map 으로 정규화된다
ee_pose	제어	/ee_pose	✔	엔드이펙터 pose. ee_pose_publisher 가 TF 에서 만들어 발행한다

변수	스택	소스	상태	설명
gripper_position	제어	joint_states 파생	 미배선	연속적인 그리퍼 위치. NO_DATA 로 응답한다
gripper_last_command_result	제어	/gripper_control/gripper_action_states		move_gripper_to_target 의 결과 채널. 이벤트성 이며 보통 직접 조회하지 않는다 (아래 참조)
session_state	수집	/session		세션/에피소드 상태 (IDLE / IN_SESSION / IN_EPISODE) 와 task label
scene_objects	제어	/scene/objects		scene 안 물체들의 종류·크기·pose. 물체 이름으로 접근하는 map 이다
scene_last_command_result	제어	/scene/command_results		reset_scene 의 결과 채널. 이벤트성 — 명령이 없으면 갱신되지 않는다
named_targets	제어	SRDF 조회 (static)		그룹별 named target 목록. 최초 조회 시 lazy 하게 가져와 캐시 한다

session_state 만 수집 스택에 묶인다 — /session 의 발행자가 rdfp 의 session_control_node 이기 때문이다. 제어 스택만 띄운 상태에서 조회하면 오류가 아니라 quality: NO_DATA 로 응답한다(정의된 변수의 값 없음은 200 + quality 로 표현한다).

각 변수의 value 형태

모두 JSON 이며, 원본 ROS 메시지 구조를 3.2 의 변환 규약에 따라 옮긴 것이다.

joint_states — sensor_msgs/JointState

```
{ "header": { "stamp": { "sec": 1786199567, "nanosec": 168720085 },
  "frame_id": "base_link" },
  "position": { "panda_joint1": 0.0, "panda_joint2": -0.785,
    "panda_finger_joint1": 0.0, "...": 0.0 },
  "velocity": { "panda_joint1": 0.0, "...": 0.0 },
  "effort": { "panda_joint1": null, "...": null } } // mock 은 토크 미발행 → null
```

panda_finger_joint1/2 가 함께 들어온다 — 연속적인 그리퍼 위치가 필요하면 여기서 읽는다.

ee_pose — geometry_msgs/PoseStamped

```
{ "header": { "stamp": { "sec": 1786199577, "nanosec": 839673204 },
  "frame_id": "panda_link0" }, // 기준 좌표계
  "pose": {
    "position": { "x": 0.30702, "y": -5.2e-12, "z": 0.59027 }, // m
    "orientation": { "x": 0.99999, "y": 0.000199, "z": -3.6e-16,
      "w": 3.46e-12 } } // x,y,z,w
```

pose 가 value.pose 아래에 한 겹 더 들어간다 (PoseStamped 구조 그대로).

session_state — rdfp_msgs/SessionCommand

```
{ "name": "session_state", "quality": "OK", "schema_version": 1,
  "value": { "header": { "stamp": { "sec": 1786935702, "nanosec": 41258 }, "frame_id": "" },
    "state": "IN_EPISODE",          // 'IDLE' | 'IN_SESSION' | 'IN_EPISODE'
    "task_label": "" } }
```

이벤트성이다 — session_control_node 가 **전이할 때만** 발행하므로 주기성이 없고 staleness 를 검사하지 않는다. 한 시간째 IN_SESSION 인 것은 낮은 값이 아니라 현재 값이다. 토픽이 TRANSIENT_LOCAL 이라 트윈이 **에피소드 도중에 늦게 붙어도 즉시 현재 상태를 받는다.**

상태를 바꾸는 것은 이 변수가 아니라 session_control_node 의 서비스다 — 트윈은 읽기만 한다.

scene_objects — rdfp_msgs/SceneObjects

```
{ "header": { "stamp": { "sec": 1786944345, "nanosec": 868439364 },
  "frame_id": "panda_link0" },          // 로봇 베이스 프레임 고정
  "objects": {
    "cube_0": { "type": "box", "dimensions": [0.05, 0.05, 0.05],
      "pose": { "position": { "x": 0.382, "y": -0.135, "z": 0.025 },
        "orientation": { "x": 0.0, "y": 0.0, "z": 0.0, "w": 1.0 } } },
    "ball_0": { "type": "sphere", "dimensions": [0.02], "pose": { "...": "..." } } }
```

배열이 아니라 map 이다. 물체 순서는 보장되지 않으므로 인덱스가 아니라 이름으로 지목한다 (joint_states 와 같은 이유). 물체가 없으면 {} 이며 그것도 "scene 이 비었다"는 유효한 상태다.

dimensions 순서는 shape_msgs/SolidPrimitive 와 같다 — box 는 x,y,z, sphere 는 반지름, **cylinder** 는 [높이, 반지름] 이다 (직관과 반대이므로 주의).

값을 채우는 것은 백엔드별 **scene 상태 노드**다. mock 은 mock_scene_state_node 이며 MoveIt planning scene 을 옮긴다. mock 계열 launch 네 개(panda_mock , panda_jgpc_mock , rdfp_panda_mock , rdfp_panda_jgpc_mock)가 이 노드를 기본으로 함께 띄운다 — 스택마다 짝이 되는 어댑터가 정해져 있어 사용자가 고를 일이 아니기 때문이다. 그러면 enable_scene_node:=false , 발행 주기는 scene_publish_rate .

mock 의 물체는 물리를 갖지 않는다. 파지에 실패해도 굴러떨어져도 pose 가 변하지 않으므로, mock 에서는 이 변수로 **성패를 관측할 수 없다.** 배관 검증용이다.

scene_last_command_result — rdfp_msgs/SceneCommandResult

```
{ "header": { "...": "..." }, "success": true, "message": "", "applied_count": 2 }
```

reset_scene 이 완료를 판정하는 데 쓰는 내부 채널이다. **연산의 outputs 를 보면 되므로 직접 조회할 일은 거의 없다.** 명령이 없는 동안 갱신되지 않는 이벤트성 값이다.

named_targets — get_all_named_targets() 의 반환값 (그룹별 목록)

```
{ "name": "named_targets", "quality": "OK", "schema_version": 1,
  "received_at": "2026-08-08T11:11:22.412Z",    // 트윈이 백엔드에서 가져온 시각
  "stamp": null,                                // ROS 헤더가 없는 값이다
  "value": { "hand": ["close", "open"],
             "panda_arm": ["extended", "ready"] } }
```

value 의 이름을 그대로 move_to_named_target 의 target 으로 넘길 수 있다.
move_group 이 아직 뜨지 않았다면 SOURCE_UNAVAILABLE + reason: "SOURCE_NODE_DOWN" 이 나오며, 백엔드가 뜬 뒤 다시 조회하면 채워진다 (트윈을 재시작할 필요는 없다).

미배선 변수 — 정의는 있으나 값이 없다. 404 가 아니다.

```
{ "name": "gripper_position", "quality": "NO_DATA", "schema_version": 1, "value": null }
```

그리퍼 명령 결과는 변수가 아니다

명령의 결과(reached_goal / stalled / effort)는 **연산 outputs 로 온다** (5.7). 트윈이 명령을 보낸 뒤 gripper_last_command_result 변수가 갱신될 때까지 기다렸다가 그 값을 옮기므로, "지금 읽은 값이 방금 보낸 명령의 결과인가"를 클라이언트가 따질 필요가 없다.

/gripper_control/gripper_action_states 토픽은 여전히 존재하지만 gripper_control_node 를 거치는 경로(teleop 등)에서만 갱신되므로 트윈의 상태원으로 쓰지 않는다.

연속적인 그리퍼 위치가 필요하면 joint_states 의 panda_finger_joint1 을 직접 읽는다.

4.2 연산

스택 열은 그 연산을 쓰려면 어느 launch 가 떠 있어야 하는지다 (1.2 참조).

- **제어** — ros2 launch robot_control panda_mock.launch.py 만으로 동작한다.
- **수집** — ros2 launch rdfp rdfp_panda_mock.launch.py 가 필요하다.

연산	스택	자원	kind	상태	설명
move_to_named_target	제어	arm	async	✓	SRDF named target(ready , extended 등)으로 이동
move_to_joints	제어	arm	async	✓	관절값 지정 이동 (joint-space)
move_linear	제어	arm	async	✓	목표 pose 까지 직선(Cartesian) 이동
move_gripper_to_target	제어	gripper	sync	✓	이름 붙은 그리퍼 목표 로 이동 (open / close / grasp). GripperCommand 액션 result 까지 기다린다
reset_scene	제어	scene + arm	sync	✓	scene 을 레시피대로 새로 만든다 . 물체 위치를 seed 로 랜덤화한다
start_session	수집	—	sync	✓	수집 세션을 연다. task_label 을 함께 설정한다
stop_session	수집	—	sync	✓	세션을 닫는다. 에피소드가 열려 있으면 함께 닫힌다
start_episode	수집	—	sync	✓	에피소드를 연다 (IN_SESSION 에서만)
stop_episode	수집	—	sync	✓	에피소드를 닫으며 성패·부가정보 를 기록에 남긴다

연산	스택	자원	kind	상태	설명
move_to_pose	제어	arm	async	 미구현	목표 pose 로 자유 계획 이동
move_gripper	제어	gripper	async	 미구현	꼭을 요청 인자로 지정. 쓸 수 있는 꼭은 설정이 정한다는 원칙이라 열지 않았다 — 필요한 꼭은 backend.targets 에 이름을 붙여 추가한다

갈리는 지점은 백엔드 노드가 어느 패키지에 있느냐 하나다. 세션/에피소드 연산 넷은 rdfp 의 session_control_node 에 중계되고, 그 노드는 rdfp_panda_mock 만 띄운다. 나머지는 robot_control 의 노드(gripper_control_node , mock_scene_state_node)나 MoveIt 을 직접 쓰므로 제어 스택만으로 충분하다.

reset_scene 이 "제어" 인 것이 헛갈릴 수 있다 — **수집을 위한 연산이지만 구현은 제어 계층에 있다.** 씬 노드가 robot_control/scene/ 으로 옮겨졌고 mock 계열 launch 넷이 모두 기본으로 띄우기 때문이다(enable_scene_node:=false 로 끌 수 있다).

수집 스택 없이 세션 연산을 부르면 PRECONDITION_FAILED + session_control_node is not available 로 거부된다. 나머지 연산은 영향을 받지 않는다.

미구현 연산을 호출하면 202 로 접수된 뒤 FAILED + EXECUTION_ABORTED 로 끝나며, 메시지에 사유가 담긴다.

아래 네 절은 구현된 연산마다 **입력 → 완료 판정 → 흔한 실패** 순으로 정리한 것이다. 완료 판정을 따로 떼어 둔 이유는, 모든 연산에 COMPLETED 가 "**목표에 도달했다**"를 뜻하지 않는 경우가 있고 그 조건이 서로 다르기 때문이다.

어느 것을 쓸 것인가

하고 싶은 것	연산
미리 정해 둔 자세(대기·수납 등)로	move_to_named_target
관절값을 이미 알고 있다 (기록 재현, 학습 정책 출력)	move_to_joints
잡은 물체를 자세 유지한 채 똑바로 옮긴다	move_linear
좌표만 알고 경로는 알아서 (자유 계획)	move_to_pose — 미구현
물체를 새 위치에 랜덤 배치한다	reset_scene
학습 데이터로 남길 구간의 시작/끝을 찍는다	start_episode / stop_episode

move_to_named_target 과 move_to_joints 는 같은 joint-space 경로이고, 목표를 SRDF 이름으로 주느냐 값으로 주느냐만 다르다.

세션/에피소드 연산 넷과 reset_scene 은 로봇을 움직이지 않는다 — **데이터 수집의 경계와 초기 조건을 만드는 연산**이며 5.10 의 수집 루프에서 함께 쓰인다.

move_to_named_target — SRDF 이름으로 이동

입력

```
{ "inputs": {
  "target": "ready",           // 필수. SRDF group_state 이름
  "velocity_scaling": 0.3,     // 0.01 ~ 1.0 (생략 시 서버 기본값)
  "max_duration_sec": 60 } }
```

쓸 수 있는 이름 먼저 확인 — 오타는 계획 단계까지 가서야 FAILED 로 돌아온다.

```
curl -s $B/variables/named_targets | python3 -m json.tool
# "value": { "hand": ["close","open"], "panda_arm": ["extended","ready"] }
```

arm 자원의 연산이므로 **panda_arm 그룹의 이름**을 쓴다 (hand 쪽 이름을 넣으면 실패한다). 이 값은 최초 조회 시 가져와 캐시된다 — 자세한 것은 3.2 ?refresh=true .

출력

```
{ "closed_loop": true,           // 항상 온다
  "final_pose":  {...},         // ee_pose 캐시의 최신값 (변수가 비었으면 생략)
  "final_joints": {...},        // joint_states 캐시의 최신값 (같음)
  "measured_age_ms": 21 }
```

이동 연산의 공통 outputs 뿐이며 이 연산 고유의 필드는 없다. 각 키의 의미, final_* 가 통째로 빠지는 조건, closed_loop: false 일 때의 해석은 4.3 에 있다.

완료 판정

202 → session_endpoint 폴링 → COMPLETED . 관절 공간 계획이라 부분 실행 개념이 없어, closed_loop: true (JTC) 이면 COMPLETED 를 도달로 봐도 된다. **closed_loop 가 false (JGPC) 면 도달 보장이 없다** (4.3).

흔한 실패

응답	원인
FAILED + EXECUTION_ABORTED , 메시지에 not found in group ... Available: [...]	target 오타이거나 다른 그룹의 이름. 메시지에 후보가 함께 온다
FAILED + EXECUTION_ABORTED , 메시지에 failed with code:	MoveIt 계획/실행 실패 (충돌·도달 불가)
400 INVALID_INPUT	target 누락 또는 빈 문자열
503 PRECONDITION_FAILED	MoveGroup 클라이언트 미준비 — /health 의 move_group 확인
409 RESOURCE_BUSY	arm 을 이미 다른 세션이 점유 (5.3 재시도)

move_to_joints — 관절값으로 이동

입력

```
{ "inputs": {
  "joints": {                                // 필수. {관절이름: 라디안}
    "panda_joint1": 0.0,
    "panda_joint2": -0.785,
    "panda_joint4": -2.356,
    "panda_joint6": 1.571 },
  "velocity_scaling": 0.3,                  // 0.01 ~ 1.0
  "max_duration_sec": 60 } }
```

넣지 않은 관절은 제약이 걸리지 않는다. 위 예처럼 4개만 주면 나머지 3개는 플래너가 알아서 정한다. 자세를 완전히 고정하려면 7개를 모두 준다.

joint_states 를 그대로 되돌려 보내지 않는다 ⚠

가장 흔한 실수다. **joint_states** 의 **position** 에는 **panda_finger_joint1/2** 가 섞여 있는데, 이들은 **panda_arm** planning group 소속이 아니라서 그대로 보내면 계획이 실패한다. 트윈은 관절 이름이 그룹에 속하는지 검사하지 않으므로(그룹의 관절 목록을 갖고 있지 않다) 400 이 아니라 **실행 단계의 FAILED** 로 나타난다.

```
current = twin.read('joint_states')['position']
arm_only = {k: v for k, v in current.items() if k.startswith('panda_joint')}
twin.run('move_to_joints', {'joints': arm_only, 'velocity_scaling': 0.2})
```

출력

```
{ "closed_loop": true,                      // 항상 온다
  "final_pose": {...},                    // ee_pose 캐시의 최신값 (변수가 비었으면 생략)
  "final_joints": {...},                  // joint_states 캐시의 최신값 (같음)
  "measured_age_ms": 21 }
```

move_to_named_target 과 같다 — **공통 outputs 뿐이고 고유 필드는 없다.**
보낸 관절값이 그대로 되돌아오지는 않으므로, 확인하려면 **final_joints** 를 읽는다 (4.3).

완료 판정

move_to_named_target 과 같다 — joint-space 계획이라 부분 실행 개념이 없고, **closed_loop: true** (JTC) 이면 **COMPLETED** 를 도달로 봐도 된다. JGPC 스택에서는 계획을 MoveIt 이 하고 실행은 명령 스트리밍이라 **open loop** 이며 도달 보장이 없다.

흔한 실패

응답	원인
400 INVALID_INPUT	joints 누락·빈 객체, 값이 숫자가 아니거나 NaN / Inf
FAILED + 메시지에 failed with code:	그룹 밖 관절 포함, 관절 한계 초과, 충돌, 도달 불가
409 / 503	move_to_named_target 과 동일

관절 한계를 넘는 값은 트윈이 막지 않는다 — MoveIt 이 계획 단계에서 거부한다.

move_linear — 직선(Cartesian) 이동

입력

```
{ "inputs": {
  "pose": {
    "position": { "x": 0.3, "y": 0.0, "z": 0.5 },
    "orientation": { "x": 1.0, "y": 0.0, "z": 0.0, "w": 0.0 }    // ROS 순서 x,y,z,w
  },
  "velocity_scaling": 0.2,
  "max_step": 0.01,
  "jump_threshold": 0.0 } }
```

인자의 의미

인자	필수	기본값	의미
pose.position	✔	—	목표 위치 [m]. 기준 좌표계는 panda_link0 (아래 참조)
pose.orientation	✔	—	목표 자세. ROS 순서 x,y,z,w 이며 단위 quaternion 이어야 한다 (norm 오차 1e-3 초과 시 400)
velocity_scaling		클라이언트 기본값	최대 속도 대비 배율(0.01~1.0). 궤적의 시간축만 늘리고 경로 모양은 바꾸지 않는다
max_step		0.01 (1 cm)	Cartesian 보간 간격 [m]. 작을수록 경로를 촘촘히 검사해 정확하지만 계획이 느려진다
jump_threshold		5.0	관절 공간 급변 차단 임계값. 인접 보간점 사이 관절 변화가 이 배수를 넘으면 경로를 거기서 끊는다. 0.0 은 검사 안 함 이라 특이점 부근에서 팔이 튕 수 있다
max_duration_sec		60	이 세션의 시간 상한 [s] . 초과하면 위치독이 동작을 멈추고 FAILED + TIMEOUT 으로 끝낸다 (9.3). MoveIt 의 계획 시간이 아니다
frame_id		—	현재 무시된다. 스키마에는 있으나 백엔드가 읽지 않는다

기준 좌표계는 항상 panda_link0 이다. MoveGroupClient 생성자의 frame_id (기본 panda_link0)가 쓰이며, 입력의 frame_id 는 전달되지 않는다. 다른 좌표계 기준으로 주고 싶으면 클라이언트가 미리 변환해야 한다.

max_step 과 jump_threshold 는 **계획 품질과 속도의 맞교환**이다. 기본값으로 두고, 경로가 자주 끊기면(FAILED 에 낮은 % 가 찍히면) max_step 을 줄여 본다.

quaternion 은 단위벡터여야 한다. 아니면 400 INVALID_INPUT 으로 **로봇에 닿기 전에** 거부된다.

범위 위반의 처리는 jsonschema 설치 여부에 달려 있다. 설치되어 있으면 velocity_scaling: 2.0 같은 값이 스키마 검증에서 400 으로 거부되고, 없으면 검증을 건너뛴 뒤 백엔드가 0.01~1.0 으로 **잘라서** 실행한다. 즉 같은 요청이 환경에 따라 400 도 되고 성공도 된다 — 클라이언트는 범위를 스스로 지키는 편이 안전하다.

출력

```
{ "closed_loop": true,           // 항상 온다
  "final_pose": {...},          // ee_pose 캐시의 최신값 (변수가 비었으면 생략)
  "final_joints": {...},        // joint_states 캐시의 최신값 (같은)
  "measured_age_ms": 21 }
```

공통 outputs 뿐이다 (4.3). 이 연산에만 있는 필드는 없으며, 특히 **계획 비율**(fraction)이 담기지 않는다 — 그래서 아래의 도달 검증이 필요하다.

완료 판정 — fraction 은 오지 않는다 ⚠

Cartesian 경로는 **장애물을 회피하지 못한다**. 목표까지 직선으로 갈 수 없으면 계획 비율(fraction)이 1.0 미만이 되는데, 트윈의 outputs 에는 **그 값이 실리지 않는다** — 백엔드(follow_trajectory_async)가 성공 시 아무 값도 돌려주지 않기 때문이다.

계획 비율	결과
< 60%	FAILED — 메시지: Path planning failed: only 43.2% of the path was planned
60 ~ 100%	COMPLETED — 그만큼만 이동하고 정상 종료. outputs 에 아무 표시가 없다
100%	COMPLETED — 목표 도달

즉 **COMPLETED 를 도달로 믿으면 안 된다**. outputs.final_pose 를 요청한 pose 와 직접 비교해야 하며, 코드는 5.6 에 있다.

임계값 60% 는 MoveGroupClient 의 DEFAULT_FRACTION_THRESHOLD 이고, move_linear 의 입력 스키마에 fraction_threshold 가 없어 **호출자가 조정할 수 없다**.

일반적인 "저 위치로 가라"는 자유 계획(move_to_pose)이 맞지만 아직 미구현이다.

move_gripper_to_target — 그리퍼 (동기)

목표를 **이름으로** 지정한다. target 문자열 하나가 필수 입력이다.

파라미터	필수	값	용도
target	✓	open close grasp	이동할 그리퍼 목표 이름

```
curl -s -X POST $B/operations/move_gripper_to_target \
-H 'Content-Type: application/json' -d '{"inputs": {"target": "open"}}'
```

outputs 에 어떤 목표를 수행했는지 target 이 함께 담긴다.

지원 목표는 서버가 알려준다 — 이름을 하드코딩하지 말고 카탈로그에서 읽는다.

```
curl -s $B/operations | jq '.operations[]
| select(.name=="move_gripper_to_target")
| .inputs_schema.properties.target.enum'
# ["close", "grasp", "open"]
```

이 enum 은 설정에 손으로 적는 값이 아니라 **backend.targets 에서 기동 시 파생된다** (그래서 이름순으로 정렬되어 온다). 목표를 늘릴 때 두 곳을 고칠 일이 없고, 둘이 어긋난 설정은 트윈이 아예 뜨지 않는다.

정의되지 않은 목표는 400 INVALID_INPUT 으로 거절되며, 메시지에 사용 가능한 목표 목록이 담긴다.

목표에 대응하는 폭과 힘은 **설정이 정한다**. 기본 제공은 셋이다.

목표	position	max_effort	의미
open	0.04	(없음)	열기

목표	position	max_effort	의미
close	0.0	(없음)	빈손으로 닫기
grasp	0.0	30.0	물체 파지. 목표는 close 와 같고 힘만 다르다

두 값은 control_msgs/GripperCommand goal 의 position / max_effort 로 그대로 들어간다.
 position 은 **손가락 사이 거리가 아니라 관절 하나(panda_finger_joint1)의 목표값**이라
 실기 franka 의 move / grasp 가 쓰는 width 와 2배 차이가 난다. max_effort 를 적지
 않으면 0 = 드라이버 기본 효과치다.

close 로 물건을 쥐려 하지 않는다. 힘이 없어 실기에서는 파지 없는 이동으로
 해석될 수 있다 — 쥔 때는 grasp 다.

출력

```
{ "target": "close", "position": 0.0182, "effort": 0.0,
  "stalled": true, "reached_goal": false }
```

이동 연산의 공통 outputs (4.3) 은 오지 않는다 — closed_loop / final_pose /
 final_joints 가 없고, 대신 그리퍼 액션 result 가 그대로 실린다. 각 키를 어떻게
 읽는지는 바로 아래에 있다.

COMPLETED 는 "액션이 끝났다"는 뜻이다

트윈은 명령을 /gripper_control/gripper_cmds 에 발행하고 **결과가 돌아올 때까지
 기다린다.** 액션을 직접 부르지 않는 이유는 **액션 goal 전송이 서비스라 rosbag2 가
 기록하지 못하기** 때문이다 — 명령이 토픽으로 흘러야 학습 데이터의 action 채널이 남는다.
 따라서 응답이 왔다는 것은 동작이 끝났다는 뜻이며, 무엇으로 끝났는지는 위 outputs 가
 말한다.

조합	해석
reached_goal: true	목표 폭에 도달했다 (빈손으로 열림/닫힘)
stalled: true	힘을 내는데 움직이지 않는다 = 물체를 물었다
둘 다 false	중단되었거나 목표에 못 미쳤다

reached_goal: false 를 실패로 읽지 않는다 ⚠ 물체를 쥐면 목표까지 갈 수 없으므로
 당연히 false 가 된다. 파지 판정의 축은 stalled 다. 트윈은 오차를 근거로 성패를
 뒤집지 않고 result 를 그대로 보고한다 (설계서 6.7). 코드는 5.7 에 있다.

흔한 실패

응답	원인
FAILED + 메시지 no publisher for topic	기동 시 퍼블리셔가 만들어지지 않았다 — backend.topic / topic_type 설정 확인
FAILED + TIMEOUT (결과 없음)	gripper_control_node 또는 컨트롤러(panda_hand_controller) 미기동
FAILED + TIMEOUT	sync_timeout_sec (기본 5초) 안에 결과가 오지 않았다. 명령은 이미 나갔으므로 결과를 모른다 — joint_states 로 확인한다

응답	원인
409 RESOURCE_BUSY	gripper 자원 점유 중 (arm 과는 독립이다 — 4.4)

reset_scene — 물체를 레시피대로 랜덤 배치

scene 을 **통째로 교체**한다. 기존 물체는 전부 제거되고 레시피가 정한 물체만 남는다.

```
curl -s -X POST $B/operations/reset_scene -H 'Content-Type: application/json' \
-d '{"inputs": {"scene": "one_cube", "seed": 42}}'
```

입력	필수	뜻
scene		레시피 이름. 쓸 수 있는 값은 GET /operations 의 scene.enum 에 있다
seed		무작위 추출 seed. 생략하면 0

출력

```
{ "scene": "one_cube", "seed": 42, "applied_count": 1,
  "objects": [ { "name": "cube_0", "type": "box",
    "dimensions": [0.05, 0.05, 0.05],
    "position": { "x": 0.377, "y": 0.104, "z": 0.025 } } ] }
```

outputs.objects 가 재현의 근거다 ⚠️ seed 만 기록하면 **추출 방식이 바뀌는 순간 재현이 깨진다**. 실제 배치를 stop_episode 의 metadata 로 넘겨 에피소드에 붙인다 (5.10 참조). 그래야 나중에 "어떤 배치에서 실패했는지" 를 조회할 수 있다.

무엇이 랜덤인지는 레시피가 정한다. 설정(backend.scenes)에서 축마다 숫자면 고정, [최소, 최대] 면 그 구간에서 균등 추출이다.

```
- { name: cube_0, type: box, size: [0.05, 0.05, 0.05],
  x: [0.35, 0.55], y: [-0.15, 0.15], z: 0.025 }
#      ↑ 랜덤      ↑ 랜덤      ↑ 고정(안착 높이)
```

레시피 추가는 **YAML 편집만으로 끝난다** — scene 의 enum 은 이 표에서 파생되므로 따로 적지 않는다 (8.2 의 규칙과 같다).

흔한 실패

증상	원인
400 INVALID_INPUT	선언되지 않은 scene 이름. 응답 메시지에 쓸 수 있는 이름이 나열된다
409 RESOURCE_BUSY	팔이 움직이는 중이다 . scene 과 arm 을 함께 잡는다 (4.4)
결과가 오지 않고 timeout	백엔드 scene 노드가 없다. mock 계열 launch 는 기본으로 띄우므로 먼저 enable_scene_node:=false 로 켜는지 확인하고, 단독으로 띄우려면 ros2 run robot_control mock_scene_state_node

scene 리셋은 에피소드 밖에서 한다. 에피소드 안에서 부르면 물체가 순간이동하는 장면이 학습 데이터에 들어간다.

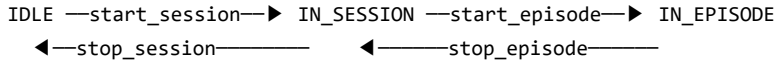
mock에서는 물체가 물리를 갖지 않는다 — 배치는 되지만 그리퍼로 잡히지 않는다.

실제 데이터 수집은 물리 백엔드에서만 성립한다.

start_session / stop_session / start_episode / stop_episode — 수집 경계

학습 데이터의 어디부터 어디까지가 한 에피소드인지를 정하는 연산이다. 트윈은

/session 을 직접 발행하지 않고 session_control_node 에 중계한다 — 상태 기계는 그 노드 하나가 갖는다.



현재 상태는 session_state 변수로 확인한다 (4.1).

start_session — task_label 을 함께 받는다.

```
curl -s -X POST $B/operations/start_session -H 'Content-Type: application/json' \
-d '{"inputs": {"task_label": "pick_red_cube"}}'
```

라벨을 별도 연산으로 두지 않은 이유는, 빈 라벨로 시작하면 그 세션의 모든 에피소드에 빈 라벨이 박히고 기록이 끝난 뒤에는 고칠 수 없기 때문이다.

stop_episode — 성과와 부가정보를 남긴다. 두 값은 기록에 실려 데이터셋의 success / metadata 가 된다.

```
curl -s -X POST $B/operations/stop_episode -H 'Content-Type: application/json' \
-d '{"inputs": {"outcome": "failure", "metadata": {"seed": 42, "scene": "one_cube"}}}'
```

입력	뜻
outcome	"success" / "failure" . 생각하면 '판정 없음'이며 실패가 아니다
metadata	JSON 객체. seed·초기 배치 등. 배열이나 스칼라는 400 이다

outcome 을 생각한 것과 "failure" 는 다르다. 전자는 판정 주체가 없었다는 뜻이고(텔레오퍼레이션 수집 등) 후자는 작업이 실패했다는 뜻이다. **파지 실패는 유효한 학습 데이터**이므로 "failure" 를 학습셋에서 무조건 빼면 안 된다.

에피소드를 닫는 것은 전적으로 클라이언트의 몫이다 ⚠ 트윈은 작업이 실패해도 stop_episode 를 자동으로 부르지 않는다. 자동 수집에서 실패는 정상 경로이므로 트윈이 판단하면 유효한 실패 에피소드를 잘라먹는다. try/finally 로 감싼다 (5.10).

흔한 실패

증상	원인
PRECONDITION_FAILED + invalid command	상태가 맞지 않는다 (IDLE 에서 start_episode 등). 재시도가 아니라 상태를 먼저 맞춰야 한다
PRECONDITION_FAILED + session_control_node is not available	노드가 안 떠 있다. ros2 run rdfp session_control_node

역등한 시작 — 이전 실행이 남긴 열린 에피소드는 `stop_session` 하나로 정리된다 (`IN_EPISODE` 에서 부르면 에피소드와 세션이 순서대로 닫힌다). 다만 `IDLE` 에서 부르면 거부되므로 상태를 먼저 본다.

```
if twin.read('session_state')['state'] != 'IDLE':
    twin.run('stop_session')
```

4.3 이동 연산의 공통 outputs

`move_to_named_target` / `move_to_joints` / `move_linear` 셋이 여기 해당한다. 셋 다 고유 출력 필드가 없어 아래가 응답의 전부다. `move_gripper_to_target` 은 해당하지 않는다 — 그리퍼 액션 `result` 를 따로 싹는다 (4.2).

```
{ "closed_loop": true,           // 항상 온다
  "final_pose":  {...},         // ee_pose 캐시의 최신값 (해당 변수가 비었으면 생략)
  "final_joints": {...},        // joint_states 캐시의 최신값 (같은)
  "measured_age_ms": 21 }       // 위 두 값 중 더 오래된 쪽의 나이
```

`final_*` 는 **상태 캐시의 최신 스냅샷을 그대로 쓴 것**이라 동작 종료 시각과 정확히 일치하지 않는다. 별도 조회를 하지 않아 지연이 없는 대신, 신선도는 `measured_age_ms` 로 직접 판단해야 한다. 해당 상태 변수가 `NO_DATA` 면 그 키 자체가 빠지므로 클라이언트는 **키 존재 여부를 확인** 해야 한다.

`fraction` 은 제공되지 않는다. `move_linear` 가 부분 경로를 실행하고도 `COMPLETED` 로 끝날 수 있는데 그 사실이 `outputs` 에 드러나지 않는다 — 4.2 의 `move_linear` 절을 반드시 읽는다.

`closed_loop: false` 이면 `COMPLETED` 가 도달을 보장하지 않는다. JGPC 스택은 명령 스트리밍(open loop)이라 정상 종료해도 목표에 도달했다는 뜻이 아니다. 이때는 `final_pose` / `final_joints` 를 목표와 비교해 직접 판단한다. 트윈 메타(`GET /robot_twins/{twin}`)에서 호출 전에 확인할 수 있다.

4.4 자원 락 — 무엇이 동시에 실행되나

자원	연산
arm	<code>move_to_named_target</code> , <code>move_to_joints</code> , <code>move_linear</code> , <code>move_to_pose</code> , <code>reset_scene</code>
gripper	<code>move_gripper_to_target</code> , <code>move_gripper</code>
scene	<code>reset_scene</code>
(없음)	<code>start_session</code> , <code>stop_session</code> , <code>start_episode</code> , <code>stop_episode</code>

- 같은 자원은 **동시에 하나만** 실행된다. 두 번째 요청은 `409 RESOURCE_BUSY` .
- arm 과 gripper 는 독립**이라 병렬 실행된다 (팔을 움직이며 그리퍼 조작 가능).
- `reset_scene` 은 scene 과 arm 을 함께 잡는다.** 물체를 순간이동시키는 동안 팔이 그 공간으로 들어오면 안 되고, 반대로 팔이 움직이는 중에 물체가 바뀌어도 안 된다. 둘 중 하나라도 점유 중이면 `409` 이며, **전부 잡히거나 하나도 안 잡힌다.**
- 세션/에피소드 연산은 자원을 잡지 않는다.** 자원 락의 수명은 연산 실행 시간뿐인데 에피소드는 `start ~ stop` 두 연산에 걸쳐 있어 락으로 보호할 수 없다. 배타 제어는 `session_control_node` 의 상태 기계가 하며, 거부는 `PRECONDITION_FAILED` 로 온다.

현재 점유 상태는 GET /resources 로 확인한다 (arm / gripper / scene).

5. 예제 프로그램

5.1 curl — 최소 흐름

```
B=http://127.0.0.1:8801/api/v1/robot_twins/panda01
```

```
# 준비 확인
```

```
curl -s $B/health | python3 -m json.tool
```

```
# 상태 조회
```

```
curl -s $B/variables/joint_states | python3 -m json.tool
```

```
# 이동 (비동기) → session_endpoint 획득
```

```
EP=$(curl -s -X POST $B/operations/move_to_named_target \
-H 'Content-Type: application/json' \
-d '{"inputs":{"target":"ready","velocity_scaling":0.3}}' \
| python3 -c 'import json,sys; print(json.load(sys.stdin)["session_endpoint"])')
```

```
# 완료까지 폴링
```

```
while ;; do
  S=$(curl -s http://127.0.0.1:8801$EP | python3 -c 'import json,sys; print(json.load(sys.stdin)["status"])')
  echo "status=$S"; [ "$S" = RUNNING ] || break; sleep 1
done
```

```
# 그리퍼 (동기)
```

```
curl -s -X POST $B/operations/move_gripper_to_target \
-H 'Content-Type: application/json' -d '{"inputs": {"target": "open"}}'
```

5.2 Python — 재사용 가능한 클라이언트

표준 라이브러리만 쓴다. ROS 가 필요 없다.

```

"""로봇 트윈 최소 클라이언트."""
from __future__ import annotations

from typing import Any, Optional

import json
import time
import urllib.error
import urllib.request

class TwinError(Exception):
    """트윈이 요청을 거부했을 때 (4xx / 5xx)."""

    def __init__(self, status: int, body: dict) -> None:
        err = body.get('error', {})
        super().__init__(f"HTTP {status} {err.get('code')}: {err.get('message')}")
        self.status = status
        self.code = err.get('code')
        self.body = body

class RobotTwin:
    def __init__(self, host: str = '127.0.0.1', port: int = 8801,
                 twin_id: str = 'panda01') -> None:
        self.base = f'http://{host}:{port}/api/v1/robot_twins/{twin_id}'
        self.root = f'http://{host}:{port}'

    # ----- 저수준 -----
    def _request(self, path: str, method: str = 'GET',
                 body: Optional[dict] = None) -> tuple[int, Any]:
        url = path if path.startswith('http') else self.base + path
        data = None if body is None else json.dumps(body).encode()
        headers = {'Content-Type': 'application/json'} if data else {}
        req = urllib.request.Request(url, data=data, headers=headers, method=method)
        try:
            with urllib.request.urlopen(req, timeout=10) as res:
                raw = res.read()
                return res.status, (json.loads(raw) if raw else None)
        except urllib.error.HTTPError as exc:
            raw = exc.read()
            parsed = json.loads(raw) if raw else {}
            if exc.code >= 400:
                raise TwinError(exc.code, parsed) from None
            return exc.code, parsed

    # ----- 상태 -----
    def health(self) -> dict:
        return self._request('/health')[1]

    def read(self, name: str, *, require_ok: bool = True) -> Any:
        """변수 값을 반환한다. quality 를 확인하지 않고 쓰지 않도록 강제한다."""
        body = self._request(f'/variables/{name}')[1]
        if require_ok and body['quality'] != 'OK':
            raise RuntimeError(

```

```

        f"variable '{name}' is {body['quality']}"
        f"{' (' + body['reason'] + ') ' if body.get('reason') else ''}"
    )
    return body['value']

def read_many(self, *names: str) -> dict:
    return self._request(f"/variables?names={','.join(names)}")[1]['values']

def resources(self) -> dict:
    return self._request('/resources')[1]

# ----- 연산 -----
def run(self, operation: str, inputs: Optional[dict] = None, *,
        poll_sec: float = 0.5, timeout_sec: float = 120.0) -> dict:
    """연산을 실행하고 종료까지 기다린다. 동기·비동기를 모두 처리한다."""
    status, body = self._request(f'/operations/{operation}', 'POST',
                                {'inputs': inputs or {}})
    if status == 200:
        # 동기 연산 - 이미 끝났다
        return body

    endpoint = body['session_endpoint'] # 202 - 폴링한다
    deadline = time.monotonic() + timeout_sec
    while True:
        state = self._request(self.root + endpoint)[1]
        if state['status'] != 'RUNNING':
            return state
        if time.monotonic() > deadline:
            raise TimeoutError(f"'{operation}' did not finish in {timeout_sec}s")
        time.sleep(poll_sec)

def cancel(self, session_endpoint: str) -> dict:
    return self._request(self.root + session_endpoint, 'DELETE')[1]

# ----- 안전 -----
def estop(self) -> dict:
    return self._request('/estop', 'POST')[1]

def release_estop(self) -> dict:
    return self._request('/estop', 'DELETE')[1]

if __name__ == '__main__':
    twin = RobotTwin()

    h = twin.health()
    print(f"move_group={h['move_group']} estop={h['estop']}")

    # value 는 원본 메시지 구조 그대로다 (3.2 / 4.1 참조).
    print('joint1 =', twin.read('joint_states')['position']['panda_joint1'])
    print('ee x,y,z=', twin.read('ee_pose')['pose']['position']) # PoseStamped → value.pose

    result = twin.run('move_to_named_target',
                     {'target': 'ready', 'velocity_scaling': 0.3})
    print('결과:', result['status'])
    if result['status'] == 'FAILED':
        print('사유:', result['error']['code'], result['error']['message'])

```

```

else:
    # closed_loop=False 면 도달 보장이 없으므로 goal_error 를 반드시 본다
    out = result.get('outputs', {})
    print('closed_loop =', out.get('closed_loop'))

```

5.3 자원 경합 처리 — 409 재시도

```

import random
import time

def run_with_retry(twin: RobotTwin, operation: str, inputs: dict,
                  *, attempts: int = 5) -> dict:
    """자원이 비기를 기다렸다가 실행한다. jitter 로 재시도 물림을 막는다."""
    for i in range(attempts):
        try:
            return twin.run(operation, inputs)
        except TwinError as exc:
            if exc.code != 'RESOURCE_BUSY' or i == attempts - 1:
                raise
            # estimated_remaining_ms 는 현재 오지 않는다. 있으면 쓰고 없으면
            # 지수 백오프로 대체한다. jitter 는 재시도 물림을 막는다.
            wait_ms = exc.body['error'].get('estimated_remaining_ms')
            delay = (wait_ms / 1000.0 if wait_ms else 2.0 ** i) + random.uniform(0, 1.0)
            print(f'arm busy, retry in {delay:.1f}s')
            time.sleep(delay)
    raise RuntimeError('unreachable')

```

5.4 취소

```

status, body = twin._request('/operations/move_to_named_target', 'POST',
                             {'inputs': {'target': 'extended', 'velocity_scaling': 0.05}})
endpoint = body['session_endpoint']

time.sleep(1.0)
print(twin.cancel(endpoint))    # 202 {"status":"RUNNING","phase":"CANCELING"}

# 취소는 즉시 확정되지 않는다 - 감속 정지 후 CANCELLED 가 된다
while True:
    state = twin._request(twin.root + endpoint)[1]
    if state['status'] != 'RUNNING':
        break
    time.sleep(0.3)
print(state['status'], state.get('outputs', {}).get('stopped_at'))

```

5.5 상태 모니터링 루프 (ETag 활용)

```
import urllib.request

url = 'http://127.0.0.1:8801/api/v1/robot_twins/panda01/variables/joint_states'
etag = None
while True:
    req = urllib.request.Request(url)
    if etag:
        req.add_header('If-None-Match', etag)
    try:
        with urllib.request.urlopen(req) as res:
            etag = res.headers.get('ETag')
            print(json.loads(res.read())['value']['position']['panda_joint1'])
    except urllib.error.HTTPError as exc:
        if exc.code == 304:
            pass # 값이 그대로다 - 본문이 오지 않는다
        else:
            raise
    time.sleep(0.2)
```

5.6 move_linear — 도달 여부를 직접 검증한다

COMPLETED 는 도달을 뜻하지 않는다 (4.2). outputs 에 fraction 이 없으므로 요청한 pose 와 final_pose 를 비교하는 것이 유일한 확인 수단이다.

```
def move_linear_verified(twin: RobotTwin, pose: dict, *, tol_m: float = 0.005) -> dict:
    """직선 이동 후 실제 도달했는지 확인한다.

    반환 dict 의 `reached` 가 False 면 경로 일부만 실행된 것이다 - 트윈은 이를
    실패로 보고하지 않으므로 클라이언트가 판단해야 한다.
    """
    result = twin.run('move_linear', {'pose': pose, 'velocity_scaling': 0.2})
    if result['status'] != 'COMPLETED':
        return {'reached': False, 'result': result}

    # 상태 변수가 비어 있으면 키 자체가 없다 (4.3).
    final = result.get('outputs', {}).get('final_pose')
    if final is None:
        return {'reached': False, 'result': result, 'why': 'no final_pose in outputs'}

    want, got = pose['position'], final['pose']['position'] # PoseStamped → .pose
    error = max(abs(want[a] - got[a]) for a in ('x', 'y', 'z'))
    return {'reached': error <= tol_m, 'error_m': error, 'result': result}

target = {'position': {'x': 0.3, 'y': 0.0, 'z': 0.5},
          'orientation': {'x': 1.0, 'y': 0.0, 'z': 0.0, 'w': 0.0}}

check = move_linear_verified(twin, target)
if not check['reached']:
    # 60~100% 만 계획되어 중간에서 멈춘 경우가 대부분이다.
    print(f"직선 경로가 막혔다 (오차 {check.get('error_m', float('nan')):.3f} m)")
```

tol_m 은 로봇·용도에 맞게 정한다. measured_age_ms 가 크면 비교 자체가 무의미하므로, 엄밀함이 필요하면 outputs.measured_age_ms 도 함께 본다.

5.7 그리퍼 — 파지 여부 판정

move_gripper_to_target 은 GripperCommand 액션의 **result** 까지 기다린 뒤 반환한다. 따라서 별도 폴링 없이 outputs 만 보면 된다.

목표는 세 가지다. close 와 grasp 는 목표 폭이 같고 **힘만 다르다** — 물건을 쥌 때는 grasp 를 쓴다.

목표	폭	힘	쓰임
open	0.04	—	열기
close	0.0	없음	빈손으로 닫기
grasp	0.0	30 N	물체를 쥌다. 닿으면 그 힘으로 버티며 멈춘다

```
def set_gripper(twin: RobotTwin, target: str) -> dict:
    """그리퍼를 조작하고 액션 결과를 그대로 돌려준다.

    반환 dict 의 판정 규칙:
        reached_goal=True           목표 폭에 도달했다 (빈손으로 닫힘/열림)
        stalled=True                힘을 내는데 움직이지 않는다 = 물체를 물었다
        둘 다 False                중단되었거나 목표에 못 미쳤다
    """

    return twin.run('move_gripper_to_target', {'target': target})['outputs']

out = set_gripper(twin, 'grasp')
if out['stalled']:
    print(f"파지 성공 - 폭 {out['position']:.4f} m, 힘 {out['effort']:.1f} N")
elif out['reached_goal']:
    print('끝까지 닫혔다 - 잡은 것이 없다')
```

stalled 이 파지 판정의 축이다. 물체를 물면 목표까지 못 가므로 reached_goal 은 False 가 된다 — 이걸 실패로 읽으면 안 된다. 트윈은 오차를 근거로 성패를 뒤집지 않고 result 를 그대로 넘긴다. grasp 로 보냈는데 reached_goal: true 가 왔다면 **손가락이 끝까지 닫혔다는 뜻이므로 잡은 것이 없다.**

mock에서는 effort 가 의미 없다. GripperActionController 는 feedback 을 발행하지 않으므로 완료 판정이 result 에만 의존하고, 힘 값도 시뮬레이션되지 않는다. 목표의 max_effort 가 실제로 의미를 갖는 것은 실기 franka 에서다 — 거기서는 max_effort = 0 (= close)이 파지 없는 이동으로 해석될 수 있어, 쥐려면 반드시 힘이 붙은 목표(grasp)를 써야 한다.

연속적인 폭 자체가 필요하면 joint_states 의 panda_finger_joint1 을 읽는다 (그 값은 손가락 사이 거리의 절반이다).

5.8 move_to_joints — 현재 자세를 읽어 되돌려 보내기

관절값 이동의 가장 흔한 쓰임은 **읽은 자세를 나중에 재현**하는 것이다. 이때 `joint_states` 를 그대로 보내면 finger joint 때문에 실패하므로 팔 관절만 걸러낸다 (4.2).

```
ARM_PREFIX = 'panda_joint'

def capture_pose(twin: RobotTwin) -> dict:
    """현재 팔 자세를 move_to_joints 입력 형태로 저장한다."""
    position = twin.read('joint_states')['position']
    return {k: v for k, v in position.items() if k.startswith(ARM_PREFIX)}

def restore_pose(twin: RobotTwin, joints: dict, *, tol_rad: float = 0.01) -> bool:
    """저장해 둔 자세로 복귀하고 실제 도달을 확인한다."""
    result = twin.run('move_to_joints', {'joints': joints, 'velocity_scaling': 0.2})
    if result['status'] != 'COMPLETED':
        print('실패:', result.get('error', {}).get('message'))
        return False

    # JGPC(open loop)에서는 COMPLETED가 도달을 보장하지 않으므로 직접 확인한다.
    final = result.get('outputs', {}).get('final_joints', {}).get('position', {})
    return all(abs(final.get(name, 1e9) - value) <= tol_rad
               for name, value in joints.items())

saved = capture_pose(twin)          # 예: 티칭 직후 저장
twin.run('move_to_named_target', {'target': 'ready'})
print('복귀:', restore_pose(twin, saved))
```

지정하지 않은 관절은 현재값으로 고정된다. 일부만 바꾸고 싶으면 그 관절만 보내면 되고, 나머지는 움직이지 않는다.

```
twin.run('move_to_joints', {'joints': {'panda_joint1': 0.5}}) # 1번 축만 움직인다
```

예전에는 그렇지 않았다. 지정한 관절에만 제약이 걸려 목표가 자세 하나가 아니라 집합이 되었고, 플래너가 그중 아무거나 골랐다. `panda_joint1` 만 준 호출이 나머지 6축을 최대 3.5 rad 움직여 엔드이펙터가 로봇 뒤쪽 위로 넘어간 것이 실측된다. 지금은 `MoveGroupClient` 가 SRDF 에서 그룹 관절을 얻고 `/joint_states` 로 현재값을 채워 **전 관절을 제약**한다. 그룹 밖 관절(`panda_finger_*`)은 채우지 않는다.

5.9 arm + gripper 순차 시퀀스

두 자원은 독립이라 서로를 막지 않는다 (4.4). 아래는 지금 구현된 연산만으로 집어 올리는 최소 흐름이다.

```

def pick(twin: RobotTwin, above: dict, grasp: dict) -> bool:
    """접근 → 하강 → 파지 → 상승. 각 단계의 결과를 확인하고 진행한다."""
    if not set_gripper(twin, 'open')['reached_goal']:
        print('그리퍼를 열지 못했다'); return False

    # 접근 지점까지는 자유 계획이 맞지만 move_to_pose 가 미구현이라 직선으로 간다.
    if not move_linear_verified(twin, above)['reached']:
        print('접근 실패'); return False
    if not move_linear_verified(twin, grasp)['reached']:
        print('하강 실패'); return False

    # 힘을 주며 달는다. 물체를 물면 stalled 로 끝나고, 끝까지 달렀다면 빈손이다 (5.7).
    if not set_gripper(twin, 'grasp')['stalled']:
        print('아무것도 잡지 못했다'); return False

    return move_linear_verified(twin, above)['reached']

```

above 와 grasp 는 move_linear 의 pose 입력 그대로다 — position (m) + orientation (쿼터니언, ROS 순서 x,y,z,w , 단위 노름). 좌표계는 **panda_link0 고정**이다.

호출부는 다음과 같다. pick 자체는 로봇 절차만 담고, 사전 점검·좌표 구성·오류 분류는 바깥에 둔다.

```
DOWN = {'x': 1.0, 'y': 0.0, 'z': 0.0, 'w': 0.0} # x축 180° - 그리퍼가 아래를 본다
```

```
def above_of(grasp: dict, *, clearance_m: float = 0.10) -> dict:
    """파지 지정 바로 위의 접근 지정. 자세는 같고 z 만 띄운다."""
    return {'position': {**grasp['position'],
                        'z': grasp['position']['z'] + clearance_m},
            'orientation': grasp['orientation']}

def main() -> int:
    twin = RobotTwin()

    # 팔 연산은 MoveGroup 이 준비되어야 시작된다. 그러려면 쓸 거라면 불필요하다 (9.3).
    health = twin.health()
    if health['move_group'] != 'READY':
        print(f"move_group 미준비: {health.get('move_group_error')}")
        return 1

    grasp = {'position': {'x': 0.40, 'y': 0.0, 'z': 0.10}, 'orientation': DOWN}

    try:
        ok = pick(twin, above_of(grasp), grasp)
    except TwinError as exc:
        # 400(입력) / 409(자원 점유·E-stop) / 503(백엔드 미준비) - 세션이 만들어지지 않았다.
        print(f'거절됨: {exc}')
        return 1
    except TimeoutError as exc:
        # 폴링 상한 초과. 로봇은 아직 움직이는 중일 수 있다 - 상태를 확인하고 판단한다.
        print(f'시간 초과: {exc} (현재 자원: {twin.resources()})')
        return 1

    print('집었다' if ok else '실패 - 중간 자세로 남아 있다')
    return 0 if ok else 1

if __name__ == '__main__':
    raise SystemExit(main())
```

이 예제는 실제로 로봇을 움직인다. grasp 좌표는 예시일 뿐이므로 자기 셋업에 맞게 바꾼다. 파지 판정(stalled)은 그 자리에 실제로 물체가 있어야 성립한다.

자원 경합이 잦은 환경이라면 pick 내부의 호출을 5.3 의 run_with_retry 로 감싼다. 지금은 409 가 그대로 TwinError 로 올라와 시퀀스가 중단된다.

단계가 늘어나면 클라이언트 조합이 아니라 복합 연산으로 옮긴다 (8.3). 위 코드는 중간에 죽으면 로봇이 어떤 자세로 남을지 클라이언트만 알고 트윈은 모른다 — 취소·복구 책임이 프로토콜 밖에 있다는 뜻이다.

5.10 자동 에피소드 수집 루프

물체 위치를 매 회 랜덤화하며 pick-and-place 를 반복해 학습 데이터를 모으는 흐름이다.

5.9 의 pick 을 그대로 쓰고, 그 바깥에 초기 조건(reset_scene)과 데이터 경계 (start_episode / stop_episode) 를 두른다.

```

reset_scene(seed=i) - move_to_named_target(ready) ┐ start_episode
                                                    | pick → place      ← 학습 데이터
                                                    └ stop_episode

```

물체 순간이동과 홈 복귀는 에피소드 밖이다. 안에 넣으면 물리적으로 불가능한 장면이 데이터에 섞인다.

```

def collect(twin: RobotTwin, episodes: int, *, scene: str = 'one_cube',
            task_label: str = 'pick_red_cube') -> None:
    """에피소드를 연속으로 수집한다. 실패도 유효한 데이터로 남긴다."""
    # 이전 실행이 남긴 열린 에피소드를 정리한다 (IDLE 에서는 거부되므로 먼저 확인).
    if twin.read('session_state')['state'] != 'IDLE':
        twin.run('stop_session')

    twin.run('start_session', {'task_label': task_label})
    try:
        for seed in range(episodes):
            placement = twin.run('reset_scene', {'scene': scene, 'seed': seed})['outputs']
            twin.run('move_to_named_target', {'target': 'ready'})

            grasp = _grasp_pose_of(placement['objects'][0])
            above = above_of(grasp)

            twin.run('start_episode')
            ok = False
            try:
                ok = pick(twin, above, grasp)
            finally:
                # 실패해도 반드시 닫는다 - 트윈은 자동으로 닫아 주지 않는다.
                twin.run('stop_episode', {
                    'outcome': 'success' if ok else 'failure',
                    # seed 가 아니라 **실제 배치**가 재현의 근거다.
                    'metadata': {'seed': seed, 'scene': scene,
                                'objects': placement['objects']},
                })
            print(f"episode {seed}: {'성공' if ok else '실패'}")
        finally:
            twin.run('stop_session')

def _grasp_pose_of(obj: dict) -> dict:
    """배치된 물체의 파지 지정. 물체 중심을 잡는다고 본다."""
    return {'position': dict(obj['position']), 'orientation': DOWN}

```

reset_scene 의 outputs.objects 를 **두 곳에 쓴다** — 파지 좌표를 만들고, 그대로 stop_episode 의 metadata 로 넘겨 에피소드에 붙인다. 후자가 없으면 나중에 "어떤 배치에서 실패했는지" 를 알 수 없다.

try/finally 가 핵심이다. pick 이 예외로 빠져나가도 에피소드는 닫혀야 한다. 열린 채로 남으면 다음 start_episode 가 거부되고, 그 구간의 데이터도 경계를 잃는다.

필요한 것: session_control_node , 백엔드 scene 노드(mock 은 mock_scene_state_node), 그리고 기록 중인 rosbag2. 셋 중 하나라도 없으면 루프는 돌지만 데이터가 남지 않는다.

앞의 둘은 `rdfp_panda_mock.launch.py` 가 함께 띄우므로 따로 실행할 필요가 없다 (scene 노드는 `enable_scene_node:=false` 로 끌 수 있다).

mock에서는 물체가 잡히지 않는다 — 물리가 없어 파지 판정(`stalled`)이 성립하지 않는다. mock 으로는 배관만 검증하고, 실제 수집은 물리 백엔드에서 한다.

6. 로봇 연동 설정

설정 파일은 `src/robot_twin/config/robot_twin_panda01.yaml` 이며, `colcon build` 시 `share/rdfp/config/` 로 설치된다.

6.1 전체 구조

```
twin:          # 트윈 식별
http:          # HTTP 바인딩
ros:           # ROS 환경
moveit:        # MoveIt 연동 (필수)
sessions:      # 세션 수명주기
variables:     # 상태 변수 목록
operations:    # 연산 목록
```

6.2 필수 설정 — 이것부터 맞춘다

```
twin:
  id: panda01          # URL 경로에 들어간다. 영숫자/-/_ 만 허용
  description: Franka Emika Panda

http:
  host: 0.0.0.0          # 노출 범위는 방화벽으로 통제한다
  port: 8801             # 트윈마다 다르게 준다
  tls: false            # true 는 아직 지원하지 않는다

ros:
  domain_id: 31          # rclpy 초기화 전에 환경변수로 적용된다
  rmw: rmw_cyclonedds_cpp
  node_name: robot_twin_panda01 # 트윈마다 고유해야 한다
  use_sim_time: false

moveit:
  move_group_mode: jtc    # 필수. 'jtc' 또는 'jgpc'
  planning_group: panda_arm
```

move_group_mode 에 `auto` 를 쓸 수 없다. 자동 판별은 토픽 그래프 조회라서 DDS 디스커버리가 안정되기 전에 호출하면 **조용히 잘못된 클라이언트**를 만든다. 증상이 "이동 명령이 아무 효과가 없다"로 나타나 추적이 어렵다. 그래서 설정 로드 시점에 거부한다.

로봇 스택	move_group_mode
rdfp_panda_mock.launch.py (JointTrajectoryController)	jtc

로봇 스택	move_group_mode
rdfp_panda_jgpc_mock.launch.py (JointGroupPositionController)	jgpc

설정 오류는 **기동 시점에** 잡힌다. ROS 초기화 전에 검증하므로 실패가 빠르고 명확하다.

```
twin config error: 1 validation error for TwinConfig
moveit.move_group_mode
  Value error, moveit.move_group_mode must be 'jtc' or 'jgpc'; 'auto' is forbidden ...
```

검증 항목에는 **연산 정의가 백엔드를 구동할 수 있는지도** 들어간다. move_gripper_to_target 에 backend.targets 가 없거나(키 오타 포함) 어떤 목표에 service / position 이 모두 빠져 있으면 트윈은 뜨지 않는다.

```
twin config error: 1 validation error for TwinConfig
  Value error, operation 'move_gripper_to_target' requires a non-empty 'backend.targets' mapping, got None
```

이 검사가 없으면 트윈은 정상 기동하고 /health 도 정상이며 카탈로그에도 연산이 보이는데 **호출만 항상 EXECUTION_ABORTED** 로 끝난다 — 원인을 설정에서 찾기 어려운 실패다.

6.3 다른 로봇에 붙이기

Panda 가 아닌 로봇이면 다음을 바꾼다.

항목	바꿀 것
moveit.planning_group	해당 로봇의 Moveit planning group
variables[].source.topic	로봇의 상태 토픽
컨트롤러 액션 이름	MoveGroupJtcClient(controller_action=...) — 기본값은 /panda_arm_controller/follow_joint_trajectory
그리퍼 액션·목표 폭	move_gripper_to_target 의 backend.action 과 backend.targets — 설정만 고치면 되고 코드 변경은 없다

컨트롤러 액션 이름은 현재 코드 기본값으로 박혀 있다. 다른 로봇을 붙이려면 rdfp/twin/runtime.py 의 create_move_group_client() 호출에 controller_action= 을 넘기도록 확장해야 한다.

7. 트윈 여러 대 운용

포트를 분리한다. 리버스 프록시는 두지 않는다 (단일 장애점이 되기 때문).

```
# 설정 파일을 복사해 twin.id / ros.node_name / http.port / ros.domain_id 를 바꾼다
ros2 run robot_twin robot_twin --config /path/to/robot_twin_panda01.yaml # :8801
ros2 run robot_twin robot_twin --config /path/to/robot_twin_panda02.yaml # :8802
```

반드시 다르게	이유
twin.id	URL 경로가 겹친다
ros.node_name	ROS 노드 이름 충돌
http.port	바인딩 충돌 (address already in use)

GET /api/v1/robot_twins 는 **항상 자기 자신 하나만** 반환한다. 트윈 레지스트리가 아니다. 여러 트윈을 한 번에 조회하는 수단은 제공하지 않는다.

8. 확장하기

8.1 새 상태 변수 추가

토픽 소스이면 YAML 만 고치면 된다. 코드 수정이 없다.

```
variables:
- name: session                                # URL 경로에 쓰이므로 영숫자/_/- 만
  source:
    type: topic
    topic: /session
    msg: rdfp_msgs/msg/SessionCommand
    qos:
      reliability: reliable
      durability: transient_local             # ⚠ 원본 토픽과 맞춰야 수신된다
      depth: 1
    staleness_ms: null                       # 이벤트성 토픽은 검사하지 않는다
    units: {}
    enums: {}                                # 정수 상수를 심볼로 바꿀 때
```

추가 후 트윈을 **재시작**한다 (무중단 리로드는 지원하지 않는다).

옵션	언제 쓰나
staleness_ms: <숫자>	주기 발행 토픽. 초과하면 STALE
staleness_ms: null	이벤트성·정적 데이터. 검사하지 않는다
projection: joint_state_map	JointState 의 name[] + position[] 병렬 배열을 map 으로
projection: scene_object_map	SceneObjects 의 objects[] 를 {물체이름: {...}} 로
enums: {필드: {정수: 심볼}}	매직넘버 대신 문자열로 노출
qos.durability: transient_local	원본이 TRANSIENT_LOCAL 일 때 필수

QoS 불일치는 조용히 실패한다. TRANSIENT_LOCAL 토픽을 VOLATILE 로 구독하면 아무것도 받지 못하고 NO_DATA 로만 보인다. 값이 안 들어오면 QoS 부터 확인한다.

static 소스는 배선되어 있다. **기동 시점에 조회하지 않고** 최초 조회 요청에서 가져오므로, 기동 로그에는 다음만 남는다.

```
variable 'named_targets': static source (get_all_named_targets) will be fetched lazily on first read
```

backend.method 에 적은 이름을 MoveGroupClient 에서 찾아 호출한다. 그 메시드가 timeout / externally_spun 키워드를 받으면 자동으로 넘긴다. 조회에 실패하면 SOURCE_UNAVAILABLE 로 보고하고 최소 5초 뒤에 다시 시도한다 — 백엔드가 트윈보다 늦게 떠도 재시작 없이 회복된다.

나머지 소스(tf / service / derived)는 스키마에는 있으나 **아직 배선되지 않았다**. 설정해도 NO_DATA 로 응답하며, 기동 로그에 다음이 남는다.

```
variable 'gripper_position': source type 'derived' is not wired yet; it will report NO_DATA
```

8.2 새 연산 추가

연산은 **YAML 선언 + 백엔드 핸들러** 두 곳이 필요하다.

1) YAML 에 선언한다.

```
operations:
- name: move_to_home
  description: >-          # 카탈로그로 나가는 설명 — 아래 참고
    팔을 홈 자세로 되돌린다. 좌표를 모를 때, 작업 시작 전 초기 자세를 고정할 때
    쓴다. 완료가 도달을 보장하지 않으므로 확인이 필요하면 ee_pose 를 읽는다.
  kind: async              # sync | async — 연산별 고정
  resource: arm             # arm | gripper | null(무점유)
  idempotent: true          # 절대 목표를 지정하면 true
  backend: { method: move_to_named_target_async }
  inputs_schema:            # JSON Schema. jsonschema 설치 시 자동 검증
    type: object
    properties:
      velocity_scaling: { type: number, minimum: 0.01, maximum: 1.0 }
    additionalProperties: false
  default_timeout_sec: 60
```

description 은 주석이 아니라 데이터다. GET /operations / GET /variables 카탈로그에 그대로 실려 나가며, LLM 에이전트(MCP 서버 등)의 도구 선택은 이 문자열에 거의 전적으로 의존한다. "무엇을 하는가"에서 끝내지 말고 **언제 부르는가**를 함께 적고, 완료가 성공을 뜻하지 않는 연산은 그 사실도 적는다 — 그러지 않으면 에이전트가 거짓 성공 위에 다음 동작을 쌓는다. 생략하면 빈 문자열이라 기동은 되지만, 그만큼 에이전트가 이름만 보고 추측하게 된다.

2) rdfp/twin/backends.py 에 핸들러를 등록한다.

```
def _move_to_home(runtime, client, op, session, timeout):
    """홈 자세로 이동한다."""
    if client is None:
        raise BackendUnavailable('MoveGroup client is not ready')

    # externally_spun=True 는 필수다 - executor 가 다른 스레드에서 spin 중이다.
    future = client.move_to_named_target_async(
        'ready', velocity_scaling=_scaling(session, 'velocity_scaling'),
        externally_spun=True
    )
    _await(future, timeout=timeout, what='move_to_home')
    return _motion_outputs(runtime, closed_loop=_is_closed_loop(runtime))

_HANDLERS['move_to_home'] = _move_to_home
```

3) 도메인 검증이 필요하다면 validate_inputs() 에 규칙을 넣는다.

JSON Schema 로 표현하기 어려운 것(quaternion 정규화, 관절 한계 등)이 여기 들어간다. 이 검증은 **세션 생성 전에** 돌아 400 으로 즉시 거부되므로, 잘못된 요청이 자원을 잠갔다 푸는 낭비가 없다.

연산 설계 시 지켜야 할 규칙

규칙	이유
절대 목표만 받는다. 상대 이동 (move_relative)을 만들지 않는다	이것이 멍등성의 근거다. 깨지면 Idempotency-Key 가 즉시 필요해진다
물리적으로 움직이면 kind: async	취소 가능성이 안전 요구사항이다. 그리퍼는 짧고 취소 실익이 없어 예외
MoveGroupClient 의 동기 메서드를 부르지 않는다	HTTP 스레드에서 부르면 executor 를 기다리다 데드락에 빠진다. *_async + externally_spun=True
구현별 전용 API 를 직접 부르지 않는다	execute_trajectory() 는 JTC 전용, stream_trajectory() 는 JGPC 전용. 공통 API 만 써야 move_group_mode 변경이 코드에 영향을 주지 않는다
idempotent 를 반드시 판정한다	카탈로그로 노출되어 클라이언트의 재시도 안전성 판단 근거가 된다

그리퍼 목표를 늘릴 때는 연산을 만들지 않는다

move_gripper_to_target 의 backend.targets 에 한 줄 추가하면 끝이다 — 코드도, 새 연산도 필요 없다.

```
targets:
  open: { position: 0.04 }
  close: { position: 0.0 }
  grasp: { position: 0.0, max_effort: 30.0 }
  pinch: { position: 0.015, max_effort: 10.0 } # 추가
```

inputs_schema 의 enum 은 기동 시 targets 에서 파생되므로 **손대지 않는다**. max_effort 는 생략하면 0 = 드라이버 기본 효과치이며, 실기 franka 에서는 0 이 파지 없는 이동으로 해석될 수 있어 물체를 쥐려면 명시한다.

8.3 복합 연산 (순차 실행)

여러 단계를 순서대로 실행해야 하면 **일반 큐를 만들지 말고 복합 연산**(예: `pick_and_place`)으로 묶는다. 중간 상태·실패 처리·취소가 그 연산 내부 책임이 되어 프로토콜이 그대로 유지된다.

9. 운영 시 알아둘 것

9.1 보안 — 현재 상태 ⚠

항목	현재
인증	없음
권한	없음. 누구든 취소·E-stop 가능
TLS	없음 (평문 HTTP)
바인딩	0.0.0.0 (모든 인터페이스)

애플리케이션 수준의 접근 통제가 전혀 없다. 통제는 망 수준(방화벽, 로봇 전용 VLAN, 물리적 접근 통제)에서 해야 한다. 사내망 너머 노출이 나 클라이언트 다수 환경으로 가면 인증·TLS·바인딩 제한을 함께 도입해야 한다.

`session_endpoint` 의 추측하기 어려운 식별자가 **현재 유일한 접근 통제 수단**이다.

9.2 비상정지의 한계 ⚠

`POST /estop` 은 **소프트웨어 정지**다. 진행 중 동작을 중단하고 새 연산을 거부한다.

```
{ "estop": "ENGAGED", "locked": true, "halted": true, "in_flight_sessions": 1 }
```

- `halted: true` — 진행 중 동작을 실제로 멈췄다.
- `halted: false` — **멈추지 못했다.** 이때는 `warning` 이 함께 오며 **로봇은 계속 움직인다.** 물리 비상정지를 써야 한다.

물리 비상정지 회로를 대체하지 않는다. 실기 적용 시 반드시 하드웨어 E-stop 을 둔다.

9.3 위치독

모든 이동 연산에 `max_duration_sec` (기본 60초)이 걸린다. 초과하면 트윈이 강제 중단하고 `FAILED + TIMEOUT` 으로 종료한다.

폴링 방식이라 트윈은 클라이언트 생존을 알 수 없다. **클라이언트가 죽어도 로봇이 계속 움직이는 것을 막는 유일한 장치**이므로 값을 너무 크게 잡지 않는다.

9.4 세션 수명주기

항목	값
종료 세션 보존	15분
세션 개수 상한	200개 (종료 세션만, LRU 축출)

보존 기간이 지나면 404 다. RUNNING 세션은 축출되지 않는다.

9.5 폴링으로 할 수 없는 것

궤적을 복원할 수 없다. 1 Hz 폴링은 50 Hz 토픽의 50 샘플 중 1개만 본다. 모니터링에는 충분하지만 "이동 중 관절 궤적을 받고 싶다"는 요구는 이 구조로 충족할 수 없다. 그런 데이터가 필요하면 rosbag 기록 + 데이터셋 파이프라인을 쓴다.

9.6 단위 변환은 하지 않는다

ROS 원본 단위(m, rad)를 그대로 노출한다. deg 나 mm 가 필요하면 클라이언트가 변환한다. 각 변수의 단위는 GET /variables 의 units 로 확인한다.

10. 트러블슈팅

증상	원인	조치
twin config not found	--config 가 상대 경로	절대 경로를 쓴다 (1.3)
address already in use	이전 트윈이 살아 있음	kill -TERM -<PGID> 로 프로세스 그룹 종료
설정 검증 오류로 기동 실패	move_group_mode 누락/ auto	jtc 또는 jgpc 명시 (6.2)
팔 연산이 503 PRECONDITION_FAILED	MoveGroup 클라이언트 미준비	/health 의 move_group_error 확인. MoveIt 스택이 떠 있는지 본다. 그리퍼 연산은 이 상태에서도 동작한다 (MoveGroup 을 쓰지 않는다)
변수가 계속 NO_DATA	① 소스 미배선 (tf / service / derived) ② QoS 불일치 ③ 토픽/타입 오타	기동 로그의 variable '...' <- ... 줄 확인. 없으면 미배선
static 변수가 SOURCE_NODE_DOWN	move_group 미기동 — 백엔드를 아직 못 부른다	MoveIt 스택 기동 후 다시 조회 하면 채워진다 (최소 5초 간격으로 재시도). 트윈 재시작 불필요
static 변수 값이 옛날 것	정적 소스는 자동 무효화하지 않는다	?refresh=true 로 강제 갱신 (3.2)
변수가 SOURCE_UNAVAILABLE / NO_PUBLISHER	발행 노드가 없다	ros2 topic info <토픽> 으로 퍼블리셔 확인
변수가 계속 STALE	발행이 끊겼거나 staleness_ms 가 너무 작다	ros2 topic hz <토픽> 으로 실제 주기 확인
이동이 409 RESOURCE_BUSY	같은 자원이 점유 중	GET /resources 로 점유자 확인. Retry-After 후 재시도
이동이 400 INVALID_INPUT	quaternion 비정규화, 필수 인자 누락	메시지에 사유가 담긴다
이동이 FAILED + PLANNING_FAILED	IK 실패/충돌/도달 불가	목표를 바꾸거나 시작 자세를 조정
COMPLETED 인데 목표에 없다	① closed_loop: false (JGPC) ② move_linear 이 부분 경로만 실행	outputs.final_pose 를 목표와 비교 (5.6). move_linear 은 60~100% 계획 시 표시 없이 부분 실행된다

증상	원인	조치
그리퍼가 COMPLETED 인데 안 움직인다	COMPLETED 는 "명령을 보냈다"는 뜻 (4.2)	joint_states 의 panda_finger_joint1 로 실제 폭 확인 (5.7). 액션 서버 미기동이면 FAILED 가 난다
폴링 중 세션이 404	트윈 재시작 가능성	완료로 가정하지 말고 상태 변수로 실제 위치 확인
이동 명령이 아무 효과 없음	move_group_mode 가 스택과 불일치	JGPC 스택에 jtc 를 준 경우 등. 6.2 표 확인

진단에 유용한 명령

```
# 트윈 전체 상태
curl -s .../health | python3 -m json.tool

# 지금 무엇이 돌고 있나
curl -s '.../operations/sessions?status=RUNNING' | python3 -m json.tool
curl -s .../resources | python3 -m json.tool

# 변수 품질 한눈에
curl -s .../health | python3 -c 'import json,sys; print(json.load(sys.stdin)["variables"])'

# ROS 쪽 확인
source ~/.ros2rc
ros2 topic info /joint_states -v      # QoS 와 퍼블리셔 수
ros2 topic hz /ee_pose                # 실제 발행 주기
```

11. 알려진 제약

항목	내용
미구현 연산	move_to_pose (자유 계획), move_gripper (임의 폭 지정 — 의도적 미개방)
미배선 소스	tf, service, derived — NO_DATA 로 응답 (topic / static 은 배선됨)
static 소스 자동 무효화	미지원. ?refresh=true 또는 트윈 재시작으로만 갱신한다
move_linear 의 fraction	outputs 에 없다. 60~100% 만 계획되면 부분 실행 후 COMPLETED 가 되며 표시가 없다. final_pose 를 직접 비교한다 (5.6)
그리퍼 폭 지정	이름 붙은 목표만 받는다. 임의 폭을 요청 인자로 주는 move_gripper 는 열지 않았다 — 필요한 폭은 설정에 이름을 붙여 추가한다 (8.2)
그리퍼 취소	미지원. move_gripper_to_target 는 동기라 취소 창이 없고, E-stop 도 진행 중인 액션 goal 을 멈추지 않는다
인증·TLS	미지원 (9.1)
Idempotency-Key	미지원. 자원 락과 절대 목표 원칙으로 중복 실행을 막는다
이벤트 push	미지원. 폴링 전용 (SSE/WebSocket 없음)
설정 리로드	미지원. 재시작해야 한다
컨트롤러 액션 이름	/panda_arm_controller/follow_joint_trajectory 로 고정 (6.3)

항목	내용
estimated_remaining_ms / Retry-After	미제공. 409 응답에 잔여 시간이 오지 않는다. 백엔드가 궤적 duration 을 세션에 채우지 않는다
progress	미제공. 세션 응답에 진행률이 오지 않는다. phase 로 국면만 알 수 있다
영상	상태 변수로 노출하지 않는다. 필요하다면 web_video_server 등을 별도로

참고 문서

- [robot_twin_design.md](#) — 설계서. 결정 근거와 이력
- [../moveit/MoveGroupClient_UserGuide.md](#) — MoveGroupClient API
- [../moveit/GripperControlNode_Guide.md](#) — 그리퍼 서비스
- [../../src/robot_control/launch/README.md](#) — 로봇 스택(제어 계열) launch 인벤토리
- [../../src/rdfp/launch/README.md](#) — 수집 계열 launch 인벤토리